NVIDIA JETSON NANO — JN30D

Guide Complet de Flash

Du formatage ext4 jusqu'au démarrage Ubuntu

MATÉRIEL UTILISÉ

Carte : NVIDIA Jetson Nano + Carrier Board Auvidea JN30D
PC hôte : Ubuntu 18.04 Live (clé USB bootable)
Stockage flash : Clé USB ≥ 16 Go formatée en ext4
Câble : USB-A → USB-C (PC → port J12 du JN30D)
Alimentation JN30D : USB-C 5V / 3A

Pourquoi ce guide est nécessaire

Le JN30D est un carrier board tiers d'Auvidea — pas le kit officiel NVIDIA. L'image officielle JetPack ne fonctionne pas sur ce carrier board car il utilise un bus I2C différent pour chaque connecteur caméra CSI. Sans le BSP Auvidea, le système se bloque au démarrage avec le message 'continue booting'.

La clé USB de flash DOIT être formatée en ext4 (système de fichiers Linux). Une clé FAT32 ou NTFS (format Windows par défaut) ne supporte pas les liens symboliques Linux ni les permissions Unix — l'extraction échoue avec des erreurs 'Cannot create symlink' et 'Input/output error'. C'est la cause de la majorité des échecs de flash.

PARTIE 1 — Formater la clé USB en ext4

Cette partie se fait dans Ubuntu Live sur le PC hôte. Ouvrir un terminal avec Ctrl+Alt+T.

ÉTAPE 1 Identifier la clé USB dans le système

Brancher la clé USB et lancer cette commande pour voir tous les périphériques :

```
lsblk
# Résultat typique :
# sda  → disque interne du PC
# sdb  → clé Ubuntu Live montée sur /cdrom
# sdc  → ta clé de flash  ← identifier par la taille
```

La clé qui vient d'être branchée est le nouveau périphérique. L'identifier par sa taille dans la colonne SIZE.

ÉTAPE 2 Démonter la clé si elle est automatiquement montée

Ubuntu monte automatiquement les clés USB dès qu'elles sont branchées. Il faut la démonter avant de formater :

```
sudo umount /dev/sdc1
# Remplacer sdc1 par le nom trouvé avec lsblk
# Si 'not mounted' → continuer quand même
```

ÉTAPE 3 Formater en ext4 — l'étape clé

Cette commande efface le contenu et formate en ext4, le seul format supportant les permissions Linux :

```
sudo mkfs.ext4 -F /dev/sdc1
# -F = forcer même si erreurs mineures
# Attendre 30-60 secondes
# Succès : 'Writing superblocks and filesystem accounting information: done'
```

□ Vérifier deux fois que c'est la bonne clé ! Le formatage efface TOUT le contenu.

ÉTAPE 4 Monter la clé et configurer les permissions

```
sudo mkdir -p /mnt/usb
sudo mount /dev/sdc1 /mnt/usb
sudo chown -R ubuntu:ubuntu /mnt/usb
sudo chmod -R 777 /mnt/usb
mount | grep sdc
# Doit afficher 'rw' (read-write) et non 'ro' (read-only) ✓
```

ÉTAPE 5 Tester que la clé accepte l'écriture

```
touch /mnt/usb/test.txt
ls /mnt/usb/
# Si test.txt apparaît → clé OK ✓
# Si 'Input/output error' → clé défectueuse, en prendre une autre
mkdir -p /mnt/usb/jetson_flash
cd /mnt/usb/jetson_flash
```

✓ La clé est prête. Les extractions se feront sans erreur de permissions ou de symlinks.

PARTIE 2 — Télécharger les 3 fichiers nécessaires

Il faut exactement 3 fichiers : le BSP NVIDIA (système de base), le filesystem Ubuntu, et le BSP Auvidea (drivers JN30D).

ÉTAPE 1 Installer les dépendances avant les téléchargements

Ces bibliothèques sont nécessaires pour l'extraction et le flash. Les installer en premier :

```
sudo add-apt-repository universe
sudo apt-get update
sudo apt-get install -y qemu-user-static libxml2-utils python3 lbzip2 bzip2 adb
fastboot
# qemu-user-static : émulation ARM pour appliquer les binaires NVIDIA dans rootfs
# libxml2-utils    : outils XML requis par le script de flash NVIDIA
# python3          : scripts de flash écrits en Python
```

```
# lbzip2 / bzip2   : décompression des archives .tbz2 (format bzip2)
# adb / fastboot   : communication avec appareils Android/Tegra en Recovery
```

 Si qemu-user-static n'est pas disponible, add-apt-repository universe l'ajoute au catalogue.

ÉTAPE 2 Fichier 1 — BSP Linux NVIDIA (L4T R32.7.1)

Le système Linux for Tegra de NVIDIA pour Jetson Nano. Taille : ~200 Mo.

```
cd /mnt/usb/jetson_flash
wget -c --show-progress \
https://developer.nvidia.com/embedded/l4t/r32_release_v7.1/t210/jetson-
210_linux_r32.7.1_aarch64.tbz2
# -c = reprendre le téléchargement si interrompu
```

 Si wget échoue, ouvrir Firefox et télécharger manuellement depuis developer.nvidia.com/embedded/downloads

ÉTAPE 3 Fichier 2 — Filesystem racine Ubuntu

Le système Ubuntu complet qui sera installé sur le Jetson. Taille : ~700 Mo.

```
wget -c --show-progress \
https://developer.nvidia.com/embedded/l4t/r32_release_v7.1/t210/tegra_linux_sample-
root-filesystem_r32.7.1_aarch64.tbz2
```

ÉTAPE 4 Fichier 3 — BSP Auvidea (le plus important)

Les drivers spécifiques au JN30D. Sans ce fichier, le Jetson ne démarre pas correctement. Taille : ~3-4 Go.

- Ouvrir Firefox dans Ubuntu Live
- Aller sur : auvidea.eu/firmware/
- Cliquer sur 'Nano, TX2 NX or Xavier NX (JN30D and JNX30D)'
- Dans Firefox → Paramètres → Téléchargements → changer vers /mnt/usb/jetson_flash/
- Télécharger le fichier .tar.xz

 Le fichier s'appelle 'JN30B' sur le site Auvidea — c'est normal, il couvre aussi le JN30D. C'est le même BSP pour toute la famille JN30.

ÉTAPE 5 Vérifier les 3 fichiers

```
ls -lh /mnt/usb/jetson_flash/
# Doit afficher 3 fichiers :
# jetson-210_linux_r32.7.1_aarch64.tbz2          ~200 Mo
# tegra_linux_sample-root-filesystem_*.tbz2       ~700 Mo
# auvidea_jn30b_*.tar.xz                          ~3-4 Go
df -h /mnt/usb
# Vérifier qu'il reste au moins 15 Go libres pour l'extraction
```

PARTIE 3 — Préparer et assembler les fichiers

Extraire les 3 fichiers dans l'ordre et les assembler. L'extraction se fait directement sur la clé ext4.

ÉTAPE 1 Extraire le BSP NVIDIA

Crée le dossier Linux_for_Tegra avec tous les outils de flash et la structure de base :

```
cd /mnt/usb/jetson_flash
tar xvjf jetson-210_linux_r32.7.1_aarch64.tbz2
# x=extraire  v=verbose  j=bzip2  f=fichier
# Crée : Linux_for_Tegra/
# Durée : 2-5 minutes
ls /mnt/usb/jetson_flash/
# Doit afficher : Linux_for_Tegra/ ✓
```

ÉTAPE 2 Extraire le Filesystem dans rootfs

Le filesystem doit aller DANS Linux_for_Tegra/rootfs/. Le sudo est obligatoire pour les permissions Unix :

```
sudo tar xvjf tegra_linux_sample-root-filesystem_r32.7.1_aarch64.tbz2 \
     -C Linux_for_Tegra/rootfs/
# -C = extraire dans ce dossier cible
# sudo obligatoire → conserve uid/gid/permissions des fichiers système
# Durée : 10-15 minutes (gros fichier)
ls /mnt/usb/jetson_flash/Linux_for_Tegra/rootfs/
# Doit afficher : bin/ boot/ etc/ home/ lib/ usr/ var/ ...
```

 Sans sudo, les permissions sont incorrectes et le Jetson ne démarrera pas. Ne jamais oublier sudo.

ÉTAPE 3 Extraire le BSP Auvidea

Le BSP Auvidea contient le kernel et les device tree spécifiques au JN30D :

```
tar xvJf /mnt/usb/jetson_flash/*.tar.xz -C /mnt/usb/jetson_flash/
# J majuscule = format xz (≠ z=gzip ≠ j=bzip2)
# Durée : 10-20 minutes (fichier de 3-4 Go)
ls /mnt/usb/jetson_flash/
# Nouveaux dossiers apparus : kernel_out/ ou similaire
```

ÉTAPE 4 Copier les fichiers Auvidea dans Linux_for_Tegra

Les fichiers kernel d'Auvidea remplacent ceux génériques NVIDIA pour que la carte reconnaisse le JN30D :

```
sudo cp -rfv /mnt/usb/jetson_flash/kernel_out/* \
             /mnt/usb/jetson_flash/Linux_for_Tegra/
# -r=récursif  -f=forcer  -v=verbose
# Adapter 'kernel_out' selon le vrai nom du dossier extrait
```

ÉTAPE 5 Appliquer les binaires NVIDIA

Ce script copie les librairies propriétaires NVIDIA dans rootfs. Obligatoire avant le flash :

```
cd /mnt/usb/jetson_flash/Linux_for_Tegra
sudo chmod +x *.sh
sudo ./apply_binaries.sh
# Installe dans rootfs/ : drivers GPU/ISP, librairies CUDA, config démarrage
# Utilise qemu-user-static pour émuler ARM et installer les paquets .deb
# Résultat attendu : 'L4T BSP package installation completed'
```

✓ Quand apply_binaries.sh se termine sans erreur, les fichiers sont prêts pour le flash.

PARTIE 4 — Mettre le JN30D en mode Recovery

Le mode Recovery est indispensable : il 'endort' le Jetson — il n'utilise plus sa mémoire eMMC et attend les instructions du PC. Sans ce mode, le PC ne peut pas écrire sur la mémoire interne.

ÉTAPE 1 Localiser le bloc J32 et les pins FR

- Regarder le dessus de la carte JN30D
- Chercher le bloc J32 — 8 contacts en cuivre en 2 rangées de 4
- Le pin numéro 1 est CARRÉ, les autres sont ronds — c'est la référence
- Retourner la carte → regarder les labels au DOS → trouver les 2 pins 'FR' (Force Recovery)

 Bien identifier les pins FR avant de court-circuiter. Les mauvais pins peuvent endommager la carte.

ÉTAPE 2 Procédure de mise en Recovery — ordre exact

 Faire ces étapes dans l'ordre exact. La carte doit être éteinte avant de court-circuiter les pins FR.

1. Débrancher le câble USB-C d'alimentation du JN30D — carte complètement éteinte
2. Court-circuiter les 2 pins FR avec un jumper ou un trombone plié
3. Brancher le câble USB-C entre le port J12 du JN30D et un port USB-A du PC Ubuntu
4. La carte s'alimente via USB-C et entre en mode Recovery automatiquement

 En mode Recovery, aucun affichage n'apparaît sur le moniteur — c'est normal. La carte attend silencieusement.

ÉTAPE 3 Vérifier que le PC détecte la carte

```
lsusb | grep NVIDIA
# Résultat attendu :
# 'Bus 00X Device 00X: ID 0955:7f21 NVIDIA Corp. APX'  ✓
# Si rien → vérifier les pins FR et le câble USB
```

✓ Si 'NVIDIA Corp. APX' apparaît → la carte est bien en mode Recovery. On peut flasher.

PARTIE 5 — Lancer le flash

ÉTAPE 1 Checklist finale avant de lancer

- ☐ Câble USB-C branché entre JN30D (J12) et PC
- ☐ lsusb montre 'NVIDIA Corp. APX'
- ☐ Tu es dans /mnt/usb/jetson_flash/Linux_for_Tegra/
- ☐ PC branché au secteur (pas en mode batterie seule)
- ☐ Mise en veille du PC désactivée
- ☐ libxml2-utils et qemu-user-static installés

 NE PAS débrancher le câble USB pendant le flash. NE PAS mettre en veille. NE PAS fermer le terminal. Durée : 10-20 minutes.

ÉTAPE 2 Lancer le script de flash

```
cd /mnt/usb/jetson_flash/Linux_for_Tegra
sudo bash flash.sh jetson-nano-emmc mmcblk0p1
# jetson-nano-emmc = profil Jetson Nano avec mémoire eMMC interne
# mmcblk0p1        = partition cible sur l'eMMC du Jetson
# Pas de flag -r → crée une nouvelle image system.img propre
```

ÉTAPE 3 Ce que tu verras défiler — tout est normal

```
Board ID(3448) version(401)
copying bctfile ... done.
copying bootloader ... done.
Making Boot image ... done.
populating rootfs ... done.
Making system.img ...
system.img built successfully.
The target has been flashed successfully.
# ↑ Cette dernière ligne = FLASH RÉUSSI ✓
```

✓ 'The target has been flashed successfully' = le flash est terminé avec succès !

PARTIE 6 — Premier boot du Jetson Nano

ÉTAPE 1 Préparer les connexions

1. Retirer le jumper FR des pins J32 — OBLIGATOIRE sinon la carte reste en Recovery
2. Débrancher le câble USB-C qui allait vers le PC
3. Connecter le moniteur via HDMI au port HDMI du JN30D
4. Connecter clavier USB et souris USB
5. Connecter le câble Ethernet (recommandé pour Internet)
6. Rebrancher l'alimentation USB-C 5V/3A → la carte démarre automatiquement

ÉTAPE 2 Wizard de configuration Ubuntu

Attendre 2-5 minutes → le wizard s'ouvre :

- Licence NVIDIA → accepter
- Langue → French ou English
- Disposition clavier → choisir selon ton clavier physique
- Fuseau horaire → Africa/Tunis
- Nom d'utilisateur + mot de passe → bien noter ces infos !
- Attendre la fin → le bureau Ubuntu s'ouvre

✓ Bureau Ubuntu visible sur le moniteur = flash réussi ! Le Jetson tourne sur le BSP Auvidea.

ÉTAPE 3 Vérifications rapides

Ouvrir un terminal sur le Jetson (Ctrl+Alt+T) :

```
cat /etc/nv_tegra_release
# Doit afficher : R32.7.x  → JetPack 4.6 ✓
uname -m
# Doit afficher : aarch64  → ARM 64-bit ✓
free -h
# Doit afficher : ~4 Go total ✓
```

PARTIE 7 — Configuration post-boot

ÉTAPE 1 Mises à jour système

```
sudo apt update
sudo apt upgrade -y
#  apt upgrade seulement — JAMAIS apt dist-upgrade
sudo apt install -y git curl wget nano python3-pip
```

 Ne jamais faire 'apt dist-upgrade' sur Jetson — cela écrase les drivers NVIDIA et casse CUDA.

ÉTAPE 2 Installer JetPack complet

Installe CUDA 10.2, cuDNN, TensorRT, OpenCV GPU. Nécessite Internet via Ethernet. Durée : 30-60 minutes.

```
sudo apt install -y nvidia-jetpack
nvcc --version
# Vérification : 'Cuda compilation tools, release 10.2' ✓
```

ÉTAPE 3 Installer Miniforge et Python 3.11

MediaPipe nécessite Python 3.11. Miniforge (conda) l'installe sans compilation :

```
wget https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-Linux-aarch64.sh
bash Miniforge3-Linux-aarch64.sh
# → Entrée pour lire la licence
# → 'yes' pour accepter les termes
# → 'yes' pour initialiser conda automatiquement
source ~/.bashrc
# Recharger le terminal pour activer conda
conda create -n gesture_env python=3.11 -y
conda activate gesture_env
# Le prompt devient : (gesture_env) user@jetson:~$
```

ÉTAPE 4 Installer les dépendances du projet

```
pip install numpy==1.26.4
# Version compatible MediaPipe + OpenCV sur aarch64
pip install opencv-python==4.10.0.84
# Version compatible avec numpy<2 (requis par mediapipe)
pip install mediapipe==0.10.18
#  Version maximale disponible pour Linux ARM64 aarch64
# Les versions supérieures n'ont pas de wheel ARM64 précompilé
python3 -c "import cv2, mediapipe as mp, numpy as np; print('Tout OK ✓')"
```

✓ Si 'Tout OK' s'affiche — le Jetson est prêt pour le projet de détection de mains !

ÉTAPE 5 Activer SSH pour travailler depuis le PC

```
sudo apt install -y openssh-server
sudo systemctl enable ssh
sudo systemctl start ssh
ip addr show eth0
# Chercher : 'inet 192.168.x.x' → c'est l'IP du Jetson
# Depuis PC Windows → PuTTY : Host=192.168.x.x  Port=22
```

 Avec VS Code + extension 'Remote SSH' sur le PC Windows, tu travailles directement sur le Jetson comme si les fichiers étaient locaux.

ÉTAPE 6 Commande de récupération après reboot

À chaque nouvelle session terminal sur le Jetson :

```
source ~/.bashrc && conda activate gesture_env
# Si conda n'est pas reconnu après reboot :
source ~/miniforge3/etc/profile.d/conda.sh && conda activate gesture_env
```

RÉSUMÉ — Toutes les commandes dans l'ordre

PC Ubuntu Live — Formatage de la clé

```
sudo umount /dev/sdc1
sudo mkfs.ext4 -F /dev/sdc1
sudo mkdir -p /mnt/usb && sudo mount /dev/sdc1 /mnt/usb
sudo chown -R ubuntu:ubuntu /mnt/usb && sudo chmod -R 777 /mnt/usb
mkdir -p /mnt/usb/jetson_flash && cd /mnt/usb/jetson_flash
```

PC Ubuntu Live — Dépendances

```
sudo add-apt-repository universe && sudo apt-get update
sudo apt-get install -y qemu-user-static libxml2-utils python3 lbzip2 bzip2 adb
fastboot
```

PC Ubuntu Live — Téléchargements

```
wget -c https://developer.nvidia.com/embedded/l4t/r32_release_v7.1/t210/jetson-
210_linux_r32.7.1_aarch64.tbz2
wget -c
https://developer.nvidia.com/embedded/l4t/r32_release_v7.1/t210/tegra_linux_sample-
root-filesystem_r32.7.1_aarch64.tbz2
# + Fichier Auvidea .tar.xz via Firefox → auvidea.eu/firmware/
```

PC Ubuntu Live — Extraction et assemblage

```
tar xvjf jetson-210_linux_r32.7.1_aarch64.tbz2
sudo tar xvjf tegra_linux_sample-root-filesystem_r32.7.1_aarch64.tbz2 -C
Linux_for_Tegra/rootfs/
tar xvJf *.tar.xz -C /mnt/usb/jetson_flash/
sudo cp -rfv kernel_out/* Linux_for_Tegra/
cd Linux_for_Tegra && sudo chmod +x *.sh && sudo ./apply_binaries.sh
```

PC Ubuntu Live — Flash

```
lsusb | grep NVIDIA                          # → NVIDIA Corp. APX ✓
cd /mnt/usb/jetson_flash/Linux_for_Tegra
sudo bash flash.sh jetson-nano-emmc mmcblk0p1
```

Jetson Nano — Post-boot

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y nvidia-jetpack
bash Miniforge3-Linux-aarch64.sh
source ~/.bashrc
conda create -n gesture_env python=3.11 -y && conda activate gesture_env
pip install numpy==1.26.4 opencv-python==4.10.0.84 mediapipe==0.10.18
python3 -c "import cv2, mediapipe as mp; print('Tout OK ✓')"
```
